

```

!pip install -U transformers datasets torch accelerate -q

from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    Trainer,
    TrainingArguments,
    DataCollatorWithPadding
)
from datasets import load_dataset
import torch
import os

# Disable W&B and unnecessary logging
os.environ["WANDB_DISABLED"] = "true"

# Step 1: Load cleaned dataset
data_path = "cleaned_alpaca_dataset.jsonl"
dataset = load_dataset("json", data_files=data_path)
dataset = dataset["train"].shuffle(seed=42).select(range(1000))

# Step 2: Tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

def tokenize_fn(batch):
    combined = [i + " " + j for i, j in zip(batch["instruction"], batch["input"])]
    return tokenizer(
        combined,
        truncation=True,
        padding="max_length", # ensures equal token lengths
        max_length=128
    )

tokenized = dataset.map(tokenize_fn, batched=True)

# Step 3: Assign scalar labels (not lists)
def create_labels(example):
    # Return integer 1 or 0 - NOT a list
    example["labels"] = 1 if len(example["output"]) > 50 else 0
    return example

tokenized = tokenized.map(create_labels)

# Step 4: Ensure correct tensor format
tokenized.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])

# Step 5: Train/eval split
split = tokenized.train_test_split(test_size=0.2)
train_ds, eval_ds = split["train"], split["test"]

# Step 6: Model
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

# Step 7: Training arguments
training_args = TrainingArguments(
    output_dir="./bert_finetune_results",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=1,
    save_strategy="no",
    logging_dir="./logs",
    report_to="none"
)

# Data collator handles any padding dynamically
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

# Step 8: Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=eval_ds,
    data_collator=data_collator,
)

# Step 9: Train + Evaluate
trainer.train()
metrics = trainer.evaluate()
print("✅ Evaluation Metrics:", metrics)

```

Map: 100% 1000/1000 [00:00<00:00, 1185.06 examples/s]

Map: 100% 1000/1000 [00:00<00:00, 4341.09 examples/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are new. You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

 [100/100 00:19, Epoch 1/1]

Step Training Loss

 [25/25 00:01]

✅ Evaluation Metrics: {'eval_loss': 0.3926927149295807, 'eval_runtime': 1.333, 'eval_samples_per_second': 150.034, 'eval_s

Start coding or [generate](#) with AI.